

BigData

Diego Silva

Crash Course, Mayo-Junio 2026

1. Introduction

En la actualidad, las organizaciones generan grandes volúmenes de información provenientes de distintas fuentes, como aplicaciones web, dispositivos móviles, sensores y sistemas transaccionales. Para aprovechar estos datos es necesario contar con procesos que permitan almacenarlos, transformarlos y ponerlos a disposición de analistas y tomadores de decisiones.

El presente proyecto tiene como objetivo construir un pipeline de datos de extremo a extremo utilizando herramientas ampliamente utilizadas en la industria, tales como PostgreSQL, Docker, Apache Airflow y DBT.

A lo largo del proyecto se implementa una arquitectura por capas (Bronze, Silver y Gold), comenzando con la generación de datos sintéticos, continuando con su almacenamiento y transformación, y finalizando con la creación de conjuntos de datos listos para análisis.

Asimismo, se introducen conceptos fundamentales de ingeniería de datos como modelado dimensional, automatización de procesos, contenerización y transformación de datos mediante código.

1.1. Objetivo del proyecto:

Construcción de un pipeline de datos utilizando PostgreSQL, modelado por capas (Bronze, Silver, Gold), aplicando principios de ingeniería de datos para la ingestión, transformación y consumo de información.

1.2. Tecnologías utilizadas:

- PostgreSQL.
- SQL
- Docker
- Docker Compose.
- Apache Airflow
- Git/GitHub

2. Sesión 2. 19/05/26

2.1. Base de datos:

Crearemos nuestra propia base de datos, lo haremos en python:

```
import random
import csv
import logging ##Permite
import uuid ##Con esto creamos identificadores unicos
import polars as pl
import pandas as pd
```

```

from faker import Faker ##Genera datos falsos: nombres, correos
from datetime import date, datetime, timedelta

#Configuracion logs
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s', #
    datefmt='%Y-%m-%d-%H:%M:%S', ##Damos formato de fecha tipo, anio/mes/dia con hr
        :min:seg
    handlers=[logging.StreamHandler()]
)

#Funcion para crear los datos
def create_data(locale: str) -> Faker: #Nos regresara locaciones/paises de donde
    nuestros queramos, en nuestro caso sera MX
    logging.info(f"Created synthetic data for {locale.split('_')[-1]} country code.
        ") #Nos
    return Faker(locale)

#Funcion para generar un registro
def generate_record(fake: Faker)-> list:
    #Aqui se generan los datos random
    person_name = fake.name()
    user_name = person_name.replace(" ", "").lower() #reemplazamos los espacios
    email = f"{user_name}@{fake.free_email_domain()}"
    personal_number = fake.ssn() #Social Security number. Para nuestro caso sera el
        numero del seguro social
    birth_date = fake.date_of_birth()
    address = fake.address().replace("\n", ", ") ##Reemplaze el salto de linea "\n"
        por ", "
    phone_numer = fake.phone_number()
    mac_address = fake.mac_address()
    ip_address = fake.ipv4() #ip version 4 fake
    clabe = fake.iban() ##El banco, para nosotros es la clabe
    accessed_at = fake.date_time_between("-1y") ##Contiene datos del anio anterior
    session_duration = random.randint(0, 36_000) ##Maximo 10 hrs (equivalente 36
        _000)
    donwload_speed = random.randint(0, 1_000)
    upload_speed = random.randint(0,800)
    consumed_traffic = random.randint(0, 2_000_000)

    return [
        person_name, user_name, email, personal_number, birth_date, address,
        phone_numer, mac_address, ip_address, clabe,
        accessed_at, session_duration, donwload_speed, upload_speed,
        consumed_traffic
    ]

def write_to_csv(file_path: str, rows: int) -> None:
    fake = create_data("es_MX")

    #Definimos los headers
    headers = [
        "person_name", "user_name", "email", "personal_number", "birth_date", "
        address", "phone_numer", "mac_address",
        "ip_address", "clabe", "accessed_at", "session_duration", "donwload_speed",
        "upload_speed", "consumed_traffic"
    ]

```

```

with open(file_path, mode="w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(headers)

    for _ in range(rows):
        writer.writerow(generate_record(fake))

#Mensaje log
logging.info(f"Written {rows} records to the csv file.")

def add_id(file_name) ->None:
    df=pl.read_csv(file_name)
    uuid_list = [str(uuid.uuid4()) for _ in range(df.height)]
    df = df.with_columns(pl.Series("unique_id", uuid_list))
    df.write_csv(file_name)
    logging.info("added UUID to the dataset.")

def update_datetime(file_name: str, run: str) -> None:
    if run == 'next':
        current_time = datetime.now().replace(microsecond=0)
        yesterday_time = str(current_time - timedelta(days=1))
        df = pl.read_csv(file_name)
        df = df.with_columns(pl.lit(yesterday_time).alias("accessed_at"))
        df.write_csv(file_name)
        logging.info("Updated accessed timestamp")

### Hasta aqui ya generamos nuestros datos, faltaria extraerlos ###
#Es parte de nuestro pipeline, ahorita lo utilizaremos como script, pero
    futuramente,
# esta base sera utilizada como un framework

if __name__ == "__main__":

    # Logging starting of the process.

    logging.info(f"Started batch processing for {date.today()}.")

    # Define the output file name with today's date.

    #output_file = f"/work_2/data_2/batch_{date.today()}.csv"

    output_file = f"batch_{date.today()}.csv" #En este caso, sse guardo dentro de
        nuestra carpeta BigData, si

    #Aplica cuando tengo el archivo en el directorio BD_DrivenPath\chapter_2\work_2

    #y un nivel abajo esta data_2

    # Define number of records: first run - 10_372; next runs random number.

    if str(date.today()) == "2026-05-27":

        records = random.randint(100_372, 100_372)

        run_type = "first"

```

```

else:

    records = random.randint(0, 1_101)

    run_type = "next"

# Generate and write records to the CSV.
write_to_csv(f"{output_file}", records)

# Add UUID to dataset.
add_id(output_file)

# Update the timestamp.
update_datetime(output_file, run_type)

# Logging ending of the process.
logging.info(f"Finished batch processing {date.today()}.")

# Leer el archivo CSV generado
df = pd.read_csv(output_file)

# Verificar cuantos datos se generaron
print(f"Cantidad de datos generados: {len(df)}")
print(f"Cantidad esperada: {records}")

if len(df) == records:
    print("Si se genero la cantidad correcta de datos.")
else:
    print("No coincide la cantidad de datos generados.")

# Imprimir los primeros 10 datos
print("\nPrimeros 10 datos:")
print(df.head(10))

```

3. Sesión 3. 21/05/26

3.1. batch procesing:

El primer proceso es la colección de los datos, hay diferentes formas y vienen en crudo, después se procesan los datos, se limpian, para que puedan ser utilizados para análisis (eliminar datos duplicados, ver qué podemos hacer con los datos faltantes, etc).

Hasta este momento ya empieza el proceso Batching, generando bloques con distintos intervalos que nosotros definimos (numero de records, año, etc.), teniendo como resultado final el proceso de el ya almacenamiento de los datos en la base de datos.

3.2. Schemas.

Aquí creamos esquemas para poder trabajar con nuestra base de datos, cargamos nuestro archivo .csv a PostgreSQL y verificamos que se haya cargado bien mandando a llamar los primeros 10 valores de nuestro

archivo, esto lo hacemos abriendo un Query tool en nuestra tabla de datos creada.

```
SELECT
*
FROM
  batch_first_load
LIMIT
10
```

	person_name character varying (100)	user_name character varying (100)	email character varying (100)	personal_number numeric	birth_date character varying (100)	address character varying (200)
1	Ing. María Elena Rodarte	ing.mariaelenarodarte	ing.mariaelenarodarte@hotmail.c...	89279145547	2014-10-16	Boulevard Solís 528 Interior 708, Vieja Re
2	Jacobo José Carlos Ram...	jacobojosécarlosramos	jacobojosécarlosramos@yahoo.c...	18897107183	2024-02-09	Ampliación Santo Tomé y Príncipe 728 In
3	Lucas Tomás Patiño	lucastomáspatiño	lucastomáspatiño@hotmail.com	42987301308	1954-02-05	Prolongación Morelos 184 395, Vieja Mor
4	Della Mercado	dellamercado	dellamercado@gmail.com	15945144002	1995-03-08	Pasaje Muñoz 409 Edif. 311 , Depto. 926,
5	Sra. Cristal Franco	sra.cristalfranco	sra.cristalfranco@gmail.com	17550372092	2018-01-28	Eje vial Norte Zamora 903 Interior 601, Vi
6	Alvaro Núñez Escobedo	alvaronúñezescobedo	alvaronúñezescobedo@hotmail.c...	87452192997	1966-10-03	Calzada Timor-Leste 294 644, San Ana M
7	Manuel Jasso Reséndez	manueljassoreséndez	manueljassoreséndez@gmail.com	15410687162	1954-04-30	Circunvalación Norte Carrera 931 Interior
8	Nadia Paulina Camacho	nadiapaulinacamacho	nadiapaulinacamacho@gmail.com	50175037915	1927-05-20	Calzada Chiapas 504 269, Nueva Indones
9	Cristal Duarte Nevárez	cristalduartenevárez	cristalduartenevárez@yahoo.com	69565466450	1994-03-06	Retorno Sur Rodríguez 964 305, San Anton
10	Flavio Bruno Contreras	flaviobrunocontreras	flaviobrunocontreras@hotmail.com	93902051783	1980-11-12	Continuación Tlaxcala 208 Edif. 826 , Dep

Figura 1: Resultado del LIMIT 10, batch first load

3.2.1. Bronze layer.

Esta nos ayudará a hacer la gran trasnormación, esta será la *staging zone*, que es donde se moderan los datos, en esta etapa es donde se hace normalización, análisis de datos, ajustar el tipo de datos (si es necesario), ajustar daros que no estén en nuestra base de datos e incluso aquellos que estén duplicados.

Para este momento vamos a volver a crear nuestra tabla, pero para ahorrarnos el hecho de crear columna por columna nuevamente, regresearémos a nuestro Schema ->'public' ->batch_first_load y le daremos clic derecho y entraremos a la parte de *scripts* ->CREATE SCRIPT, donde nos arrojará lo siguiente:

```
-- Table: public.batch_first_load

-- DROP TABLE IF EXISTS public.batch_first_load;

CREATE TABLE IF NOT EXISTS public.batch_first_load
(
    person_name character varying(100) COLLATE pg_catalog."default",
    user_name character varying(100) COLLATE pg_catalog."default",
    email character varying(100) COLLATE pg_catalog."default",
    personal_number numeric,
    birth_date character varying(100) COLLATE pg_catalog."default",
    address character varying(200) COLLATE pg_catalog."default",
    phone character varying(100) COLLATE pg_catalog."default",
    mac_address character varying(100) COLLATE pg_catalog."default",
    ip_address character varying(100) COLLATE pg_catalog."default",
    clabe character varying(100) COLLATE pg_catalog."default",
    accessed_at time without time zone,
    session_duration integer,
    download_speed integer,
    upload_speed integer,
    consumed_traffic integer,
    unique_id character varying(100) COLLATE pg_catalog."default"
)
```

```
TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.batch_first_load
  OWNER to postgres;
```

Y nos lo llevamos a donde vamos a crear nuestra misma tabla, en nuestro caso vamos para la **bronze layer**, abrimos un nuevo **Query tool** y pegamos lo copiado (y modificado), ahora ya solo falta cargar nuestra información a la tabla.

Observación

Ojo: cambiaremos public por bronze_layer

Importante

Si por accidente creamos una tabla en un schema donde no la queríamos, para borrarla simplemente nos vamos a la tabla, abrimos un **Query tool** y ponemos el comando:

```
DELETE FROM "schema.el_nombre_de_la_tabla";
```

Ahora, ya que tenemos todo lo que necesitamos para poder trabajar con nuestra **bronze_layer**, queremos ver cuántos valores nulos hay en la tabla, para eso, abrimos nuevamente un **Query tool** y ponemos el siguiente comando:

```
SELECT
  COUNT (*) AS null_values
FROM
  bronze_layer.batch_first_load
WHERE
  person_name IS NULL
  OR user_name IS NULL
  OR email IS NULL
  OR personal_number IS NULL
  OR birth_date IS NULL
  OR address IS NULL
  OR phone IS NULL
  OR mac_address IS NULL
  OR ip_address IS NULL
  OR clabe IS NULL
  OR accessed_at IS NULL
  OR session_duration IS NULL
  OR download_speed IS NULL
  OR upload_speed IS NULL
  OR consumed_traffic IS NULL
  OR unique_id IS NULL
```

null_values	
bigint	
1	0

Figura 2: valores nulos

Para nuestro caso, nos regresa que tenemos 0 valores nulos.

Ahora queremos saber cuántos datos duplicados tenemos, para ello lo haremos con el siguiente Query tool, y nos dira si es que hay filas que cuentan con la misma información.

```
SELECT *,
COUNT (*) AS duplicate_values
FROM
bronze_layer.batch_first_load
GROUP BY (
person_name, user_name, email, personal_number, birth_date, address,
phone, mac_address, ip_address, clabe, accessed_at, session_duration,
download_speed, upload_speed, consumed_traffic, unique_id
)
HAVING COUNT (*) > 1
```

```
Successfully run. Total query runtime: 919 msec.
0 rows affected.
```

Figura 3: Salida de valores duplicados

Nuevamente, para nuestro caso nos salieron 0 filas repetidas, y solo nos mostrará el resultado del conteo si éste excede 1

Ahora, en nuestra *tabla Silver* vamos a trabajar con un diagrama estrella como el que sigue...

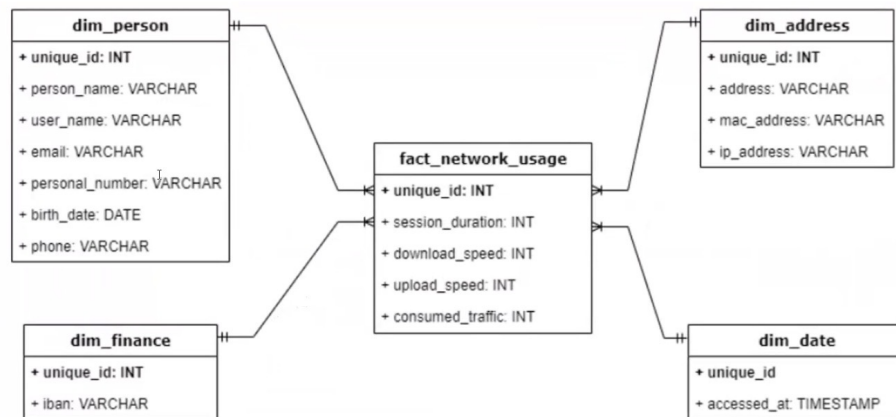


Figura 4: Diagrama estrella

Donde nuestra tabla principal será **fact_network_usage** y tendremos cuatro tablas de ahí; **dim_person**, **dim_finance**, **dim_address**, **dim_date**

Sabiendo esto, creemos nuestras tablas...

3.2.2. Silver Layer.

Creamos un nuevo esquema, una vez aquí, abrimos un Query en tablas, y empezamos a crearlas.

```
#Creamos la tabla en Silver con los valores desde bronze
```

```
CREATE TABLE
silver_layer.dim_address AS
SELECT
unique_id,
address,
mac_address,
ip_address
FROM
bronze_layer.batch_first_load
---
```

```
CREATE TABLE
silver_layer.dim_date AS
SELECT
unique_id,
accessed_at
FROM
bronze_layer.batch_first_load
---
```

```
CREATE TABLE
silver_layer.dim_finance AS
SELECT
unique_id,
clabe
FROM
bronze_layer.batch_first_load
---
```

```
CREATE TABLE
silver_layer.dim_person AS
SELECT
unique_id,
person_name,
user_name,
email,
phone,
birth_date,
personal_number
FROM
bronze_layer.batch_first_load
```

```
--- Creamos nuestra ultima tabla ---
```

```
CREATE TABLE
silver_layer.fact_network_usages AS
SELECT
unique_id,
session_duration,
download_speed,
upload_speed,
consumed_traffic
FROM
bronze_layer.batch_first_load
```


Nota

Cada tabla es para:

- `dim_address`: Esta tabla concentra información acerca de la ubicación y huella digital del usuario.
- `dim_date`: Esta tabla concentra información acerca de la hora en la que se accedió al servicio.
- `dim_finance`: Esta tabla concentra la clave interbancaria de cada usuario.
- `dim_person`: Esta tabla concentra la información relacionada con los usuarios, así evitamos redundancia y facilitamos el análisis posterior.
- `fact_network_usages`: Esta tabla almacena las métricas de uso de red y funciona como tabla de hechos dentro del esquema estrella.

Observación

En nuestra **bronze layer** realizamos la ingesta y exploración inicial de los datos, identificando posibles inconsistencias como valores nulos y registros duplicados, manteniendo siempre la información original para garantizar trazabilidad.

En la **silver layer** realizamos el modelado dimensional de los datos, construyendo tablas de dimensiones y tablas de hechos bajo un esquema estrella, lo que permite organizar la información para su posterior análisis y explotación.

— spoiler —

En la **golden layer** generamos métricas e indicadores de negocio orientados al análisis y la toma de decisiones. En esta capa se realizan cálculos agregados, como el monto diario de facturación de cada usuario, permitiendo disponer de información lista para reportes, dashboards y análisis estratégicos.

4. Sesión 4. 26/05/26

4.1. Golden layer.

Este esquema, generalmente se refiere a una capa de datos procesados, limpios y listos para consumo final.

Ahora, vamos a crear la capa *Gold*, donde ya trabajaremos los datos para distintos casos de uso, por nuestra parte, meteremos un cálculo del monto diario del pago de facturación de cada usuario y eso lo ocupará el departamento de finanzas. Adicionalmente, crearemos otra tabla para el reporte de fallas en la conexión a internet para el departamento de soporte. Y crearemos otra tabla con datos normalizados de ppi y no ppi.

Para las tablas de *financial Data*, contendrá los calculo de las siguientes columnas y una columna adicional llamada *payment_amount* que almacenará el valor calculado del pago para cada uno de los usuarios que tenemos, además incluiremos el `unique_id` para poder identificarlos a cada uno.

En SQL...

```
CREATE TABLE
golden_layer.payment_data AS
SELECT
fnu.unique_id,
df.clabe,
fnu.download_speed,
fnu.upload_speed,
fnu.sessn_duration,
fnu.consumed_traffic,
```

```
((fnu.download_speed + fnu.upload_speed + 1)/2) +
(fnu.consumed_traffic / (fnu.session_duration + 1)) AS payment_amount
FROM
silver_layer.fact_network_usages fnu
JOIN
silver_layer.dim_finance df
ON
fnu.unique_id = df.unique_id
```

En ese Query, lo que hacemos es El comando `CREATE TABLE golden_layer.payment_data AS ...` toma el resultado de la consulta (`SELECT`) y lo guarda de forma persistente en una nueva tabla. Esto significa que, una vez ejecutado, tendrás una tabla física nueva con esos datos.

Y podemos ver que efectivamente los datos ya se encuentren en nuestra tabla que hemos creado.

```
SELECT
*
FROM
golden_layer.payment_data
LIMIT
10
```

Ahora, crearemos una tabla que nos permita vincular el rendimiento de la red (`download_speed`, `upload_speed`, etc.) con datos específicos del hardware y ubicación (`address`, `mac_address`, `ip_address`).

```
CREATE TABLE
golden_layer.technical_data AS
SELECT
fnu.unique_id,
da.address,
da.mac_address,
da.ip_address,
fnu.download_speed,
fnu.upload_speed,
ROUND((fnu.session_duration/60), 1) AS min_session_duration,
CASE
    WHEN fnu.download_speed < 50 OR fnu.upload_speed < 30 OR
    fnu.session_duration/60 < 1 THEN true
    ELSE false
END AS technical_issue
FROM
silver_layer.fact_network_usage fnu
JOIN
silver_layer.dim_address da
ON
fnu.unique_id = da.unique_id
```

Por último, crearemos dos tablas, una de información personal y una de no datos personales. Recordemos que las consultas son:

`golden_layer → silver_layer → bronze_layer`, lo que no se puede hacer es `golden_layer ↗ bronze_layer`

Con el siguiente Query lo que haremos será crear una tabla `non_pii_data` que recibirá datos de la silver (que son los datos limpios pero crudos) que son los que 'enmascaremos'

```

CREATE TABLE
golden_layer.non_pii_data AS
SELECT
'***MASKED***' AS person_name,
SUBSTRING(dp.user_name, 1, 5) || '*****' user_name,
SUBSTRING(dp.email, 1, 5) || '*****' AS email,
'***MASKED***' AS personal_number,
'***MASKED***' AS birth_date,
'***MASKED***' AS address,
'***MASKED***' AS phone,
SUBSTRING(da.mac_address, 1, 5) || '*****' AS mac_address,
SUBSTRING(da.ip_address, 1, 5) || '*****' AS ip_address,
SUBSTRING(df.clabe, 1, 5) || '*****' AS clabe,
dd.accessed_at,
fnu.session_duration,
fnu.upload_speed,
fnu.consumed_traffic,
fnu.unique_id
FROM
silver_layer.fact_network_usages fnu
INNER JOIN
silver_layer.dim_address da ON fnu.unique_id = da.unique_id
INNER JOIN
silver_layer.dim_date dd ON da.unique_id = dd.unique_id
INNER JOIN
silver_layer.dim_finance df ON dd.unique_id = df.unique_id
INNER JOIN
silver_layer.dim_person dp ON df.unique_id = dp.unique_id

```

Ahora, crearemos una tabla ppi, que es una tabla normalizada y contendrá todas las columnas que sean necesarias.

Hasta éste momento, ya construimos todo el *pipeline* de forma manual de todo el proceso que nosotros vamos a estar automatizando.

★ A forma de resumen:

Generamos la base de datos → PostgreSQL → Bronze Layer → Silver Layer → Gold Layer

★ Función de cada capa:

- ★ Bronze layer:
 - Recibe los datos originales.
 - Conserva la información sin modificaciones significativas.
 - Permite mantener trazabilidad sobre la fuente de datos.
- ★ Silver layer:
 - Limpia y organiza la información.
 - Aplica transformaciones.
 - Implementa un modelo estrella mediante tablas de hechos y dimensiones.
- ★ Golden layer:
 - Genera métricas e indicadores de negocio.
 - Produce conjuntos de datos listos para análisis.
 - Facilita la construcción de reportes y dashboards.

Transición hacia la automatización:

Hasta este momento, todas las transformaciones fueron realizadas manualmente mediante consultas en PostgreSQL. Aunque este enfoque es útil para entender mejor el flujo de datos, en el mundo laboral resulta necesario automatizar estas tareas.

Para resolver este problema, se incorporarán herramientas como *Docker*, *Apache Airflow* y *DBT*, las cuales permitirán contenerizar el entorno de trabajo, orquestar procesos y ejecutar transformaciones reproducibles de forma automática.

4.2. DOCKER

Docker es una plataforma de software que permite desarrollar, empaquetar, distribuir y ejecutar aplicaciones de manera rápida y consistente. Para ello utiliza unidades estandarizadas llamadas **contenedores**, que incluyen todo lo necesario para que una aplicación funcione correctamente: código fuente, bibliotecas, dependencias, herramientas, configuraciones y entorno de ejecución (*runtime*).

La principal ventaja de Docker es que garantiza que una aplicación se comporte de la misma manera en distintos entornos, ya sea en una computadora personal, un servidor o en la nube. Esto facilita el despliegue (*deployment*), la portabilidad y la escalabilidad de las aplicaciones.

Por otro lado, *Docker Compose* es una herramienta que permite definir y administrar múltiples contenedores que trabajan de manera conjunta. Su configuración se realiza mediante un archivo llamado `docker-compose.yml`, donde se especifican los servicios, redes y volúmenes necesarios para la aplicación.

Mientras que Docker se encarga de crear y ejecutar contenedores individuales, *Docker Compose* simplifica la gestión de varios contenedores relacionados. Por ejemplo, una aplicación puede requerir un contenedor para la base de datos, otro para una aplicación desarrollada en Python y otro para Airflow. Con *Docker Compose*, todos estos servicios pueden iniciarse y administrarse mediante un solo comando: `docker compose up`

Esta separación de servicios facilita el mantenimiento y la resolución de problemas. Es decir, si ocurre un fallo en la base de datos, basta con revisar el contenedor correspondiente, sin necesidad de inspeccionar el resto de la infraestructura. De esta forma, cada componente permanece aislado, pero puede comunicarse con los demás para que la aplicación funcione como un sistema integrado.

En resumen, Docker proporciona el entorno para ejecutar aplicaciones dentro de contenedores, mientras que *Docker Compose* permite coordinar y administrar múltiples contenedores de manera sencilla, facilitando el desarrollo, las pruebas y el despliegue de aplicaciones complejas.

Otro componente importante del ecosistema Docker es *Docker Desktop*. Se trata de una aplicación que proporciona una interfaz gráfica para facilitar la creación, ejecución y administración de contenedores e imágenes Docker desde una computadora personal. Además de incluir el motor de Docker, ofrece herramientas para monitorear contenedores, gestionar imágenes, redes y volúmenes, así como acceder a registros y configuraciones del entorno de trabajo.

La principal ventaja de *Docker Desktop* es que simplifica muchas tareas que normalmente se realizarían mediante la línea de comandos, permitiendo a los desarrolladores administrar sus proyectos de manera más intuitiva y eficiente. De esta forma, es posible visualizar el estado de los contenedores, iniciar o detener servicios y supervisar recursos desde una única interfaz.

Por otro lado, *Docker Hub* es un servicio en la nube que funciona como un repositorio central para almacenar y distribuir imágenes Docker. Su propósito es facilitar el intercambio de imágenes entre desarrolladores, organizaciones y la comunidad de software de código abierto (*open source*).

A través de *Docker Hub*, los usuarios pueden buscar, descargar y reutilizar imágenes ya preparadas para diferentes tecnologías y servicios, como bases de datos, servidores web, herramientas de análisis de datos o aplicaciones completas. Esto evita tener que construir todas las imágenes desde cero y acelera significativamente el desarrollo y despliegue de aplicaciones. Asimismo, los desarrolladores pueden publicar sus propias imágenes para compartirlas con otros usuarios o mantener repositorios privados para proyectos específicos.

Otro concepto fundamental dentro de la ingeniería de datos es *Apache Airflow*. Se trata de una plataforma de código abierto diseñada para crear, programar y monitorear flujos de trabajo (*workflows*) mediante código. Al definir estos procesos como código, resulta más sencillo mantenerlos, probarlos, versionarlos y compartirlos entre distintos miembros de un equipo.

La pieza central de Airflow son los *Directed Acyclic Graphs* (DAGs), o grafos acíclicos dirigidos. Un DAG representa un conjunto de tareas y las dependencias existentes entre ellas, especificando el orden en que deben ejecutarse. Gracias a esta estructura, Airflow puede coordinar procesos complejos de manera automática y garantizar que cada tarea se ejecute únicamente cuando sus dependencias hayan sido completadas correctamente.

Dentro de Airflow existe un componente denominado *Scheduler*, encargado de programar la ejecución de los DAGs y distribuir las tareas entre los distintos *workers*. De esta manera, es posible automatizar procesos que requieren múltiples etapas y ejecutarlos de forma periódica o bajo determinadas condiciones.

En términos prácticos, Airflow permite organizar y automatizar pipelines de datos. Por ejemplo, un flujo de trabajo podría incluir la extracción de datos desde una fuente externa, su limpieza y transformación, el entrenamiento de un modelo de aprendizaje automático y la carga de los resultados en una base de datos o sistema de análisis. Cada una de estas actividades se modela como una tarea dentro de un DAG.

Además, Airflow proporciona una interfaz web que permite visualizar el estado de ejecución de los DAGs, monitorear tareas, identificar errores y analizar las dependencias entre los distintos componentes del flujo de trabajo. Si una tarea falla, las tareas dependientes no se ejecutarán hasta que el problema sea corregido.

Los DAGs suelen definirse mediante código Python. En ellos se especifican tanto las tareas como las relaciones de dependencia existentes entre ellas. Para implementar la lógica de cada tarea, Airflow utiliza elementos llamados *operadores* (*operators*). Cada operador representa una acción específica dentro del pipeline, como ejecutar código Python, transferir datos entre sistemas, ejecutar consultas SQL o interactuar con servicios externos.

Por último, otra herramienta relevante en los procesos modernos de ingeniería de datos es *Data Build Tool* (DBT). Esta herramienta se enfoca en la transformación de datos dentro de los almacenes de datos (*data warehouses*), permitiendo que dichas transformaciones se definan mediante código SQL y sean gestionadas de forma estructurada y reproducible.

El objetivo principal de DBT es transformar datos crudos en conjuntos de datos limpios, organizados y listos para el análisis. Gracias a ello, analistas, científicos de datos y usuarios de negocio pueden acceder a información confiable mediante consultas SQL sencillas, sin necesidad de comprender toda la complejidad técnica de los procesos de ingeniería de datos que existen detrás.

Ahora, instalaremos nuestro archivo `docker-compose`, para esto lo podemos hacer de forma directa entrando en el enlace o simplemente haciéndolo desde nuestra terminal el VSCode o mejor desde la GitBash, lo instalaremos directamente en una carpeta, para nuestro caso será `'src_3'` y lo haremos mediante el siguiente comando `curl -LfO 'https://airflow.apache.org/docs/apache-airflow/2.10.2/docker-compose.yaml'` Trabajaremos sobre el esqueleto que ya tenemos que nos da el `docker-compose`, solamente modificaremos algunas líneas:

5. Sesión 5. 04/06/2026

5.1. Automatización y Transformación con Docker y DBT

5.1.1. estructura del proyecto

Crearemos una infraestructura basada en contenedores de Docker para ejecutar PostgreSQL, Airflow y DBT de forma reproducible y portable:

```
src_3/
|
|-- Dockerfile
|
```

```

|-- docker-compose.yml
|
|-- requirements.txt
|
'-- dbt/
    |
    |-- dbt_project.yml
    |
    |-- profiles.yml
    |
    '-- models/
        |
        |-- source.yml
        |
        |-- staging_dim_address.sql

```

5.1.2. Dockerfile:

Crearemos un archivo *Dockerfile*, que en palabras simples, es un archivo de texto donde escribimos **paso a paso** cómo modificar una imagen existente para crear el proyecto, al final **Docker** lee este texto y construyó una **nueva imagen** exclusiva para nuestras necesidades.

```

1 # Use the official Airflow image as template
2 FROM apache/airflow:2.10.2
3
4 # Switch to airflow user to install packages
5 USER airflow
6
7 # Copy the requirements file into the container
8 COPY requirements.txt .
9
10 # Install all dependencies from the requirements file
11 RUN pip install -r requirements.txt
12
13 # Copy dbt directory into the container
14 COPY dbt /opt/airflow/dbt

```

5.1.3. Docker Compose:

Si solo necesitáramos levantar un solo contenedor, usaríamos un comando sencillo en terminal, pero nuestra arquitectura nos pide levantar varios contenedores: Postgres, Redis, el Webserver de Airflow, etc..., entonces el archivo `docker-compose` es el mapa general de la infraestructura.

5.1.4. DBT:

En la ingeniería de Datos, como ya habíamos mencionado en la introducción, existe un concepto llamado ETL, **dbt** se enfoca únicamente a la parte T (transformación).

Anteriormente hacíamos transformaciones manuales con comandos completos;

`CREATE TABLE golden_layer.payment_data AS ...`, pero hacer esto en la vida laboral resuelta muy tedioso cuando tenemos demasiadas tablas.

Es decir, con el dbt solo nos preocuparemos por escribir las consultas (SELECT) en un archivo `.sql` y el dbt se encargará automáticamente de crear o actualizar la tabla en base a los datos.

En palabras más simples, **dbt** entra a la base de datos de **PostgreSQL**, toma los datos crudos de la *Bronze layer*, ejecuta los archivos SQL para limpiarlos (Silver layer) y finalmente calcula las métricas listas para análisis (Golden layer), por ejemplo, el monto de facturación.

■ dbt_project.yml:

```
1 name: 'dbt_driven_data' ##nombre de nuestro proyecto
2 version: '1.0'
3 profile: 'default' ##Aqui se le va a decir que coneccion usar
4 model-paths: ["models"] ##Le indicamos donde estan los modelos de SQL con esta
  instruccion
```

■ profiles.yml:

Aquí le decimos a DBT a qué base de datos se conectará.

```
1 #Este archivo va a definir como dbt se va a conectar a nuestra base de datos
2 default:
3   target: dev
4   outputs: #Aqui configuramos los entornos de salida que vamos a tener
5     dev:
6       type: postgres
7       host: postgres
8       user: airflow
9       password: airflow
10      dbname: airflow
11      schema: driven_raw
12      port: 5432
```

■ source.yml:

Este archivo es muy importante porque es definir las capas que ya habíamos construido manualmente.

```
1 version : 2
2
3 sources:
4   - name: raw_source #Bronze layer
5     database: airflow
6     schema : driven_raw
7     tables:
8       - name: raw_batch_data #Es la batch_first_load
9
10  - name: staging_source #Silver layer
11    database: airflow
12    schema: driven_staging
13    tables:
14      - name: dim_address
15      - name: dim_date
16      - name: dim_finance
17      - name: dim_person
18      - name: fact_network_usages
19
20  - name: trusted_source #Gold layer
21    database: airflowschema
22    schema: driven_trusted
23    tables:
24      - name: payment_data
25      - name: technical_data
26      - name: non_pii_data
27      - name: pii_data
```

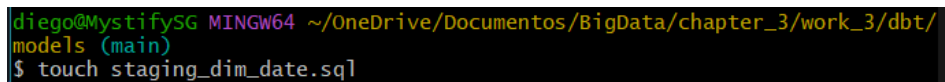
silver_layer:

- staging_dim_address.sql :
Este es el primer modelo DBT

```
1 -- Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
   tabla en nuestra capa staging y se llamara dim_address
2 {{ config(
3     -- Creamos fisicamente la tabla
4     materialized='table',
5     schema='staging',
6     alias='dim_address', -- nombre de la tabla
7     tags=['staging'] -- permite ejecutar modelos por grupo
8 ) }}
9
10 WITH source_data AS (
11     SELECT
12         unique_id,
13         address,
14         mac_address,
15         ip_address
16     FROM
17         {{source('raw_source', 'raw_batch_data')}} -- Asi fue como los
           definimos en source-yml
18 )
19
20     SELECT *
21     FROM source_data
```

Aquí, de nuestra capa silver estamos creando nuestra primera tabla `dim_address` con sus respectivas columnas, y así tenemos que hacer lo mismo para nuestras demás tablas, por lo que, desde nuestra terminal crearemos con un comando nuestra siguiente tabla, así podremos visualizarla en VSCode.

- staging_dim_date.sql:



```
diego@MystifySG MINGW64 ~/OneDrive/Documentos/BigData/chapter_3/work_3/dbt/
models (main)
$ touch staging_dim_date.sql
```

Figura 5: creamos desde la terminal "staging_dim_date"

— En VSCode:

```
1 #Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
   tabla en nuestra capa staging y se llamara dim_date
2 {{ config(
3     materialized='table', ##Creamos fisicamente la tabla
4     schema='staging',
5     alias='dim_date', #nombre de la tabla
6     tags=['staging'] #permite ejecutar modelos por grupo
7 ) }}
8
9 WITH source_data AS (
10     SELECT
11         unique_id,
12         accessed_at,
13     FROM {{source('raw_source', 'raw_batch_data')}}
14 )
15     SELECT *
```



```
16 FROM source_data
```

■ staging_dim_finance.sql:

Seguiremos la misma estructura que en la captura de pantalla (figura 5) para crear el archivo dim_finance.
— en VSCode:

```
1 #Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
  tabla en nuestra capa staging y se llamara dim_finance
2 {{ config(
3     materialized='table', ##Creamos fisicamente la tabla
4     schema='staging',
5     alias='dim_finance', #nombre de la tabla
6     tags=['staging'] #permite ejecutar modelos por grupo
7 ) }}
8
9 WITH source_data AS (
10     SELECT
11         unique_id,
12         clabe
13     FROM
14         {{source('raw_source', 'raw_batch_data')}}
15 )
16     SELECT *
17     FROM source_data
```

■ staging_dim_person.sql:

Seguiremos la misma estructura que en la captura de pantalla (figura 5) para crear el archivo dim_person.
— En VSCode:

```
1 #Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
  tabla en nuestra capa staging y se llamara dim_person
2 {{ config(
3     materialized='table', ##Creamos fisicamente la tabla
4     schema='staging',
5     alias='dim_person', #nombre de la tabla
6     tags=['staging'] #permite ejecutar modelos por grupo
7 ) }}
8
9 WITH source_data AS (
10     SELECT
11         unique_id,
12         person_name,
13         user_name,
14         email,
15         phone,
16         birth_date,
17         personal_number
18     FROM
19         {{source('raw_source', 'raw_batch_data')}}
20 )
21     SELECT *
22     FROM source_data
```

Ya solo nos faltaría nuestra tabla de hechos (fact_network_usages)

- `staging_fact_network_usages.sql`
Creamos nuestro archivo como lo hemos ido haciendo (ver figura 5).
— En VSCode:

```

1  #Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
   tabla en nuestra capa staging y se llamara staging_fact_network_usages
2  {{ config(
3      materialized='table', ##Creamos fisicamente la tabla
4      schema='staging',
5      alias='fact_network_usages', #nombre de la tabla
6      tags=['staging'] #permite ejecutar modelos por grupo
7  ) }}
8
9  WITH source_data AS (
10     SELECT
11         unique_id,
12         session_duration,
13         download_speed,
14         upload_speed,
15         consumed_traffic
16     FROM
17         {{source('raw_source', 'raw_batch_data')}}
18 )
19
20     SELECT *
   FROM source_data

```

Con esto, hemos creado ya todo lo relacionado a nuestro `silver_layer`, por lo cual nos faltaría otras tablas adicionales (`payment_data`, `non_pii_data`, `pii_data` y `technical_data`.) pero que corresponden a la siguiente capa.

Seguimos trabajando dentro de nuestra carpeta `models`.

Golden_layer:

- `trusted_non_pii_data`:
Creamos nuestro archivo como lo hemos ido haciendo (ver figura 5).
Para el Script nos basaremos completamente en los anteriores que hicimos pero cambiará un poco, principalmente porque ya no estamos en *staging* sino en *trusted*, para los atributos nos guiaremos en los scripts que ya habíamos hecho en `.sql` (ver **Sesión 4**)

Corrección de dependencias dbt: `source()` → `ref()`

Los modelos de la capa `trusted` estaban utilizando:

```
1  {{ source('staging_source', 'fact_network_usages') }}
```

para acceder a tablas generadas por otros modelos dbt.

Esto provocaba errores en el `localhost` como:

```
1  relation "driven_staging.fact_network_usages" does not exist
```

porque `source()` está diseñado para tablas externas ya existentes en la base de datos, no para modelos dbt generados dentro del proyecto.

Solución:

Se reemplazó el uso de `source()` por `ref()` para referenciar modelos de la capa `staging`.

Regla general en dbt

`source()` → tablas externas o datos de entrada definidos en `sources.yml`.

`ref()` → modelos dbt creados dentro del mismo proyecto.

— En VSCode:

```
1 #Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
   tabla en nuestra capa staging y se llamara dim_date
2 {{ config(
3     materialized='table', ##Creamos fisicamente la tabla
4     schema='trusted',
5     alias='non_ppi_data', #nombre de la tabla
6     tags=['trusted'] #permite ejecutar modelos por grupo
7 ) }}
8
9 WITH source_data AS (
10     #Como le pusimos alias a nuestras tablas, ahora le tenemos que poner los
       alias que le corresponden
11     SELECT
12         dp.person_name,
13         dp.user_name,
14         dp.email,
15         dp.personal_number,
16         dp.birth_date,
17         da.address,
18         dp.phone,
19         da.mac_address,
20         da.ip_address,
21         df.clabe,
22         dd.accessed_at,
23         fnu.session_duration,
24         fnu.upload_speed,
25         fnu.consumed_traffic,
26         fnu.unique_id,
27     FROM
28         {{ ref('staging_fact_network_usages') }} fnu
29     INNER JOIN
30         {{ ref('staging_dim_address') }} da
31         ON fnu.unique_id = da.unique_id
32     INNER JOIN
33         {{ ref('staging_dim_date') }} dd
34         ON da.unique_id = dd.unique_id
35     INNER JOIN
36         {{ ref('staging_dim_finance') }} df
37         ON dd.unique_id = df.unique_id
38     INNER JOIN
39         {{ ref('staging_dim_person') }} dp
40         ON df.unique_id = dp.unique_id
41 )
42     SELECT *
43     FROM source_data
```

■ `trusted_payment_data`:

Creamos nuestro archivo como lo hemos ido haciendo (ver figura 5).

— En VSCode:

```
1 #Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
   tabla en nuestra capa staging y se llamara dim_date
2 {{ config(
3     materialized='table', ##Creamos fisicamente la tabla
4     schema='trusted',
5     alias='payment_data', #nombre de la tabla
6     tags=['trusted'] #permite ejecutar modelos por grupo
```

```

7 ) }}
8
9 WITH source_data AS (
10     #Como le pusimos alias a nuestras tablas, ahora le tenemos que poner los
11     alias que le corresponden
12     SELECT
13         fnu.unique_id,
14         df.clabe,
15         fnu.download_speed,
16         fnu.upload_speed,
17         fnu.session_duration,
18         fnu.consumed_traffic,
19         #como para payment_data hicimos un calculo, lo que haremos es que le
20         pasaremos el calculo
21         ((fnu.download_speed + fnu.upload_speed + 1) / 2 + (fnu.
22             consumed_traffic / fnu.session_duration)) as payment_amount
23 FROM
24     {{ ref('staging_fact_network_usages') }} fnu
25 JOIN
26     {{ ref('staging_dim_finance') }} df
27     ON fnu.unique_id = df.unique_id
28 )
29
30 SELECT *
31 FROM source_data

```

■ trusted_ppi_data:

Cremos nuestro archivo como lo hemos ido haciendo (ver figura 5).

— En VSCode:

```

1 #Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
2   tabla en nuestra capa staging y se llamara dim_date
3   {{ config(
4       materialized='table', ##Creamos fisicamente la tabla
5       schema='trusted',
6       alias='ppi_data', #nombre de la tabla
7       tags=['trusted'] #permite ejecutar modelos por grupo
8   ) }}
9
10 WITH source_data AS (
11     #Como le pusimos alias a nuestras tablas, ahora le tenemos que poner los
12     alias que le corresponden
13     SELECT
14         dp.person_name,
15         dp.user_name,
16         dp.email,
17         dp.personal_number,
18         dp.birth_date,
19         da.address,
20         dp.phone,
21         da.mac_address,
22         da.ip_address,
23         df.clabe,
24         dd.accessed_at,
25         fnu.session_duration,
26         fnu.download_speed,
27         fnu.upload_speed,
28         fnu.consumed_traffic,
29         fnu.unique_id
30 FROM

```

```

29     {{ ref('staging_fact_network_usages') }} fnu
30     INNER JOIN
31     {{ ref('staging_dim_address') }} da
32     ON fnu.unique_id = da.unique_id
33     INNER JOIN
34     {{ ref('staging_dim_date') }} dd
35     ON da.unique_id = dd.unique_id
36     INNER JOIN
37     {{ ref('staging_dim_finance') }} df
38     ON dd.unique_id = df.unique_id
39     INNER JOIN
40     {{ ref('staging_dim_person') }} dp
41     ON df.unique_id = dp.unique_id
42 )
43     SELECT *
44     FROM source_data

```

■ trusted_technical_data:

Creemos nuestro archivo como lo hemos ido haciendo (ver figura 5).

— En VSCode:

```

1  #Su funcion es leer los datos desde nuestra tabla fuente y va a crear una
2  #tabla en nuestra capa staging y se llamara dim_date
3  {{ config(
4      materialized='table', ##Creamos fisicamente la tabla
5      schema='trusted',
6      alias='technical_data', #nombre de la tabla
7      tags=['trusted'] #permite ejecutar modelos por grupo
8  ) }}
9  WITH source_data AS (
10     #Como le pusimos alias a nuestras tablas, ahora le tenemos que poner los
11     #alias que le corresponden
12     SELECT
13         fnu.unique_id,
14         da.address,
15         da.mac_address,
16         da.ip_address,
17         fnu.download_speed,
18         fnu.upload_speed,
19         #para min_session... tambien hicimos un calculo, por lo que se lo
20         #pasaremos
21         round.(fnu.session_duration/60, 1) as min_session_duration,
22         case when download_speed < 50 or upload_speed < 30 or (fnu.
23             session_duration/60) < 1 then true
24         else false end as technical_issue
25     FROM
26     {{ ref('staging_fact_network_usages') }} fnu
27     JOIN
28     {{ ref('staging_dim_address') }} da
29     ON fnu.unique_id = da.unique_id

```

NOTA IMPORTANTE

- Los comentarios se hacen con - - no con #, ya que estamos trabajando en SQL
- **NO** podemos comentar dentro de (...)

6. Sesión 6. 09/06/26

...

En las sesiones anteriores se construyó la infraestructura necesaria para almacenar y transformar los datos mediante PostgreSQL, Docker y DBT. Una vez preparado el entorno, el siguiente paso consiste en **automatizar** la ejecución del pipeline.

El objetivo de esta sesión es **desplegar el entorno de Apache Airflow mediante Docker** y comprender los componentes fundamentales que intervienen en la automatización de un pipeline de datos.

¿Qué es Apache Airflow?

Apache Airflow es una plataforma de código abierto diseñada para crear, programar y monitorear flujos de trabajo. Estos flujos se representan mediante DAGs (Directed Acyclic Graphs), los cuales describen el orden en que deben ejecutarse las diferentes tareas que conforman un proceso.

Airflow no transforma ni almacena datos directamente; su función principal consiste en coordinar la ejecución de procesos y garantizar que cada tarea se ejecute en el momento adecuado y respetando las dependencias definidas por el usuario.

Debido a estas características, Airflow se ha convertido en una de las herramientas más utilizadas dentro de la ingeniería de datos para automatizar procesos ETL, pipelines analíticos y tareas de mantenimiento de sistemas.

Componentes principales de Airflow

- **Webserver:** proporciona la interfaz gráfica desde la cual se monitorean los DAGs y las ejecuciones.
- **Scheduler:** determina cuándo debe ejecutarse cada tarea según la programación establecida.
- **Worker:** ejecuta las tareas enviadas por el scheduler.
- **Metadata Database:** almacena información sobre DAGs, ejecuciones y estados de las tareas.
- **DAG:** representación del flujo de trabajo y de las dependencias existentes entre las tareas.

Para iniciar los servicios se construyó la imagen personalizada definida en el Dockerfile y posteriormente se levantaron los contenedores especificados en el archivo docker-compose.yaml.

```
diego@MystifySG MINGW64 ~/OneDrive/Documentos/BigData/chapter_3/work_3
(main)
$ docker-compose up --build
```

Figura 6: Docker compose up

NOTA: Si deseamos parar aquí la práctica por cualquier razón, basta con dar el siguiente comando para detener los contenedores pero los **conserva**

Después podemos volver a levantarlos con

Los volúmenes se convierten, salvo que usemos `docker-compose down -v`

Una vez que levante bien lo que haremos será abrir **Docker desktop** y revisar que efectivamente ya tengamos los contenedores(7), así mismo deberíamos poder observar las imágenes (7).

Y en volúmenes, debemos ver un volumen: `work_3_postgres-db-volume`.

Una vez teniendo todo esto, ya podemos ir a cualquier navegador (el de tu preferencia) e insertar `localhost:8080` directamente en la barra de navegación, la cual nos abrirá la página de *Apache Airflow*, nos pedirá usuario y contraseña (que nosotros para fines prácticos definimos 'airflow' para ambos casos).

Aquí ya estará toda la navegación de nuestra página. Ahora bien, nos dirigimos a la sección de **Admin** → **connections**

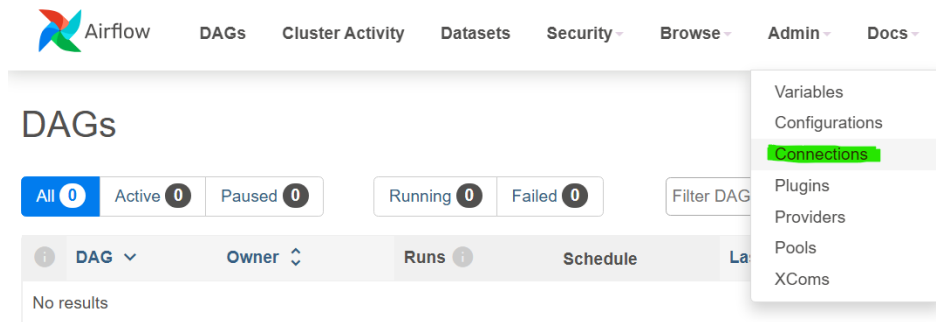


Figura 7: La sección **Connections** permite configurar las credenciales necesarias para conectarse a servicios externos como bases de datos, APIs y sistemas de almacenamiento.

Una vez aquí, crearemos una nueva conexión que tendrá como atributos:

The image shows the 'Add Connection' form in the Apache Airflow web interface. The form has the following fields: 'Connection Id' (text input, value: 'postgres_conn'), 'Connection Type' (dropdown menu, value: 'Postgres', with a note: 'Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.'), 'Description' (text area), 'Host' (text input, value: 'postgres'), 'Database' (text input, value: 'airflow'), 'Login' (text input, value: 'airflow'), 'Password' (password input, value: '*****'), and 'Port' (text input, value: '5432').

Figura 8: atributos de `postgres_conn`

Recordando que el usuario y contraseña serán los que definimos para airflow.

Una conexión almacena información necesaria para establecer comunicación con sistemas externos, como bases de datos PostgreSQL, servicios de almacenamiento, APIs o plataformas en la nube. Entre los parámetros más comunes se encuentran el nombre del host, puerto, usuario, contraseña y esquema de conexión.

El principal beneficio de este mecanismo es que las credenciales permanecen separadas del código de los DAGs. De esta forma, los desarrolladores pueden reutilizar conexiones en múltiples flujos de trabajo sin exponer información sensible directamente en los scripts de Python.

En el proyecto desarrollado durante este curso, las conexiones fueron utilizadas para permitir la comunicación entre Airflow y los diferentes servicios desplegados mediante Docker Compose, facilitando la automatización de las tareas de extracción, transformación y carga de datos.

Ahora, en la carpeta `chapter_3/work_3/dags` crearemos el `driven_data_pipeline.py` que no es más que la copia de nuestro primer archivo python `batch_generator.py` solo que e estará modificado, a continuación muestro el archivo `.py` ya modificado.

```

import random
import csv
import logging ##Permite
import uuid ##Con esto creamos identificadores unicos
import polars as pl
import pandas as pd

from faker import Faker ##Genera datos falsos: nombres, correos
from datetime import date, datetime, timedelta
##Traemos los operadores de Airflow
from airflow import DAG
import airflow.operators.bash import BashOperator
import airflow.operators.python import PythonOperator
##Traemos los operadores de SQL
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator

#Configuracion logs ##Queda igual
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s', #
    datefmt='%Y-%m-%d-%H:%M:%S', ##Damos formato de fecha tipo, anio/mes/dia con hr
    :min:seg
    handlers=[logging.StreamHandler()])

#Funcion para crear los datos ##Queda igual
def create_data(locale: str) -> Faker: #Nos regresara locaciones/paises de donde
    nuestros queramos, en nuestro caso sera MX
    logging.info(f"Created synthetic data for {locale.split('_')[-1]} country code.
    ") #Nos
    return Faker(locale)

#Funcion para generar un registro ##Queda igual
def generate_record(fake: Faker)-> list:
    #Aqui se generan los datos random
    person_name = fake.name()
    user_name = person_name.replace(" ", "").lower() #reemplazamos los espacios
    email = f"{user_name}@{fake.free_email_domain()}"
    personal_number = fake.ssn() #Social Security number. Para nuestro caso sera el
    numero del seguro social
    birth_date = fake.date_of_birth()
    address = fake.address().replace("\n", ", ") ##Reemplaze el salto de linea "\n"
    por ", "
    phone_number = fake.phone_number()
    mac_address = fake.mac_address()
    ip_address = fake.ipv4() #ip version 4 fake
    clabe = fake.iban() ##El banco, para nosotros es la clabe
    accessed_at = fake.date_time_between("-1y") ##Contiene datos del anio anterior
    session_duration = random.randint(0, 36_000) ##Maximo 10 hrs (equivalente 36
    _000)
    download_speed = random.randint(0, 1_000)
    upload_speed = random.randint(0,800)
    consumed_traffic = random.randint(0, 2_000_000)

    ##Queda igual
    return [
        person_name, user_name, email, personal_number, birth_date, address,

```



```

        phone_numer, mac_address, ip_address, clabe,
        accessed_at, session_duration, donwload_speed, upload_speed,
        consumed_traffic
    ]

#Eliminamos los parametros (lo que esta entre parentesis)
def write_to_csv() -> None:
    fake = create_data("es_MX")

    #Definimos los headers ##Queda igual
    headers = [
        "person_name", "user_name", "email", "personal_number", "birth_date", "
        address", "phone_numer", "mac_address",
        "ip_address", "clabe", "accessed_at", "session_duration", "donwload_speed",
        "upload_speed", "consumed_traffic"
    ]

    #Agregamos un if para las filas basandonos en la fecha
    if str(date.today()) == "2026-06-09"
        rows = random.randint(100_372, 100_372)
    else
        rows = random.randint(0, 1_101)

    ##Agregamos la ruta de airflow en en lugar del file_path de nuestra computadora
    local
    with open("/opt/airflow/data/raw_data.csv", mode="w", encoding="utf-8", newline
    = "") as file:
        writer = csv.writer(file)
        writer.writerow(headers)

        for _ in range(rows):
            writer.writerow(generate_record(fake))

    #Mensaje log ##Queda igual
    logging.info(f"Written {rows} records to the csv file.")

##Elinamos los parametros
##Cambiamos la ruta por nuestra ruta airflow
def add_id() ->None:
    df=pl.read_csv("/opt/airflow/data/raw_data.csv")
    uuid_list = [str(uuid.uuid4()) for _ in range(df.height)]
    df = df.with_columns(pl.Series("unique_id", uuid_list))
    df.write_csv("/opt/airflow/data/raw_data.csv")
    logging.info("added UUID tothe dataset.")

##Eliminamos los parametros
##Cambiamos la ruta por nuestra ruta airflow
def update_datetime() -> None:
    if run == 'next':
        current_time = datetime.now().replace(microsecond=0)
        yesterday_time = str(current_time - timedelta(days=1))
        df = pl.read_csv("/opt/airflow/data/raw_data.csv")
        df = df.with_columns(pl.lit(yesterday_time).alias("accessed_at"))
        df.write_csv("/opt/airflow/data/raw_data.csv")
        logging.info("Updated accessed timestamp")

### Hasta aqui ya generamos nuestros datos, faltaria extraerlos ###
#Es parte de nuestro pipeline, ahorita lo utilizaremos como script, pero
    futuramente,
#esta base sera utilizada como un framework

```

```

##Agregamos una nueva funcion y borramos el if
def save_raw_data():
    # Logging starting of the process.

    logging.info(f"Started batch processing for {date.today()}.")

    # Define the output file name with today's date.
    #output_file = f"/work_2/data_2/batch_{date.today()}.csv"
    ##Agregamos
    _write_to_csv()
    _add_id()
    _update_datetime()
    logging.info(f"finish_batch_proccesed {date.today()}")

    #y un nivel abajo esta data_2

    # Define number of records: first run - 10_372; next runs random number.

    if str(date.today()) == "2026-05-27":

        records = random.randint(100_372, 100_372)

        run_type = "first"

    else:

        records = random.randint(0, 1_101)

        run_type = "next"

    # Generate and write records to the CSV.

    write_to_csv(f"{output_file}", records)

    # Add UUID to dataset.

    add_id(output_file)

    # Update the timestamp.

    update_datetime(output_file, run_type)

    # Logging ending of the process.

    logging.info(f"Finished batch processing {date.today()}.")

    # Leer el archivo CSV generado
    df = pd.read_csv(output_file)

    # Verificar cuantos datos se generaron
    print(f"Cantidad de datos generados: {len(df)}")
    print(f"Cantidad esperada: {records}")

    if len(df) == records:

```

```

        print("Si se genero la cantidad correcta de datos.")
    else:
        print("No coincide la cantidad de datos generados.")

# Imprimir los primeros 10 datos
print("\nPrimeros 10 datos:")
print(df.head(10))

```

Una vez desplegado Apache Airflow, definamos un DAG (Directed Acyclic Graph) encargado de automatizar el flujo completo de procesamiento de datos.

El DAG implementado coordina las siguientes etapas:

1. Definición de parámetros generales del DAG.
2. Generación de datos sintéticos mediante Python.
3. Creación del esquema Bronze dentro de PostgreSQL.
4. Carga automática del archivo CSV generado.
5. Ejecución de modelos DBT para construir la capa Silver y la capa Gold.
6. Dependencias.

Gracias a esta configuración, el pipeline puede ejecutarse automáticamente respetando las dependencias entre tareas y garantizando que cada etapa se complete correctamente antes de iniciar la siguiente.

En código:

1. Definición de parámetros generales del DAG.

```

# Aqui se definen parametros que seran compartidos por todas las tareas.
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'retries': 0
}

## Definicion del DAG principal.
# Este DAG sera responsable de ejecutar el pipeline completo
# desde la extraccion de datos hasta la ejecucion de modelos DBT.
dag = DAG(
    'extract_raw_data_pipeline',
    default_args = default_args,
    description = 'DataDriven Main Pipeline.',
    schedule_interval = "* 7 * * *",
    start_date = datetime(2024, 9, 22),
    catchup = False,
)

```

Esta parte se definen los parámetros generales que utilizará Apache Airflow durante la ejecución del pipeline. Se especifica el propietario del flujo de trabajo, la programación de ejecución y la cantidad de reintentos permitidos.

Después se crea el DAG principal, el cual será responsable de coordinar todas las tareas del pipeline.

2. Generación de datos sintéticos.

```

extract_raw_data_task = PythonOperator(
    task_id = 'extract_raw_data',
    python_callable = save_raw_data,
    dag = dag,
)

```

Esta es la primer tarea del flujo que consiste en ejecutar la función `save_raw_data`, encargada de generar y almacenar los datos sintéticos utilizados durante el proyecto. Es importante mencionar que esta tarea representa la etapa inicial de la extracción de información dentro del pipeline.

— Ahora, vamos con las tareas de SQL —

3. Creamos la capa Bronze.

```
## Crea el esquema driven_raw en PostgreSQL.
### OJO, este esquema representa la capa Bronze del proyecto ###
create_raw_schema_task = SQLExecuteQueryOperator(
    task_id = 'create_raw_schema',
    conn_id = 'postgres_conn',
    sql = 'CREATE SCHEMA IF NOT EXISTS driven_raw;',
    dag = dag,
)

## Crea la tabla raw_batch_data donde se almacenaran
# los datos sin transformar provenientes del CSV.
create_raw_table_task = SQLExecuteQueryOperator(
    task_id = 'create_raw_table',
    conn_id = 'postgres_conn',
    sql = """
        CREATE TABLE IF NOT EXISTS driven_raw.raw_batch_data(
            person_name VARCHAR(100),
            user_name VARCHAR(100),
            email VARCHAR(100),
            personal_number NUMERIC,
            birth_date VARCHAR(100),
            address VARCHAR(100),
            phone VARCHAR(100),
            mac_address VARCHAR(100),
            ip_address VARCHAR(100),
            clabe VARCHAR(100),
            accessed_at TIMESTAMP,
            session_duration INT,
            download_speed INT,
            upload_speed INT,
            consumed_traffic INT,
            unique_id VARCHAR(100),
        );
    """,
    dag = dag,
)
```

Estas tareas son las responsables de contruir la capa *Bronze* dentro de **PostgreSQL**. Primero creamos el squema `driven_raw` y posteriormente la tabla `raw_batch_data`, donde se almacenarán los datos sin transformar generados previamente.

4. Carga de datos.

```
## Carga el CSV generado previamente dentro de la tabla raw_batch_data
# utilizando COPY.
load_raw_data_task = SQLExecuteQueryOperator(
    task_id = 'load_raw_data',
    conn_id = 'postgres_conn',
    sql = """
        COPY driven_raw.raw_batch_data(
            person_name, user_name, email, personal_number,
            birth_date, address, phone, mac_address, ip_address,

```

```

        clabe, accessed_at, session_duration, download_speed,
        upload_speed, consumed_traffic, unique_id
    )
    FROM '/opt/airflow/data/raw_data.csv'
    DELIMITER ','
    CSV HEADER;
    """
)

```

Una vez que hemos creado la estructura de almacenamiento, utilizamos COPY para importar automáticamente el archivo .csv generado anteriormente. De esta forma, los datos quedan disponibles dentro de la capa *Bronze* para posteriormente transformarlos.

5. Ejecución de modelos DBT

```

## Run dbt
## Ejecuta los modelos DBT etiquetados como 'staging' de la capa Silver.
run_dbt_staging_task = BashOperator(
    task_id = 'run_dbt_staging',
    bash_command = 'set -x; cd /opt/airflow/dbt && dbt run --select tag:
                    staging',
)

## Ejecuta los modelos DBT de la capa Trusted (Gold).
# Los datos quedan listos para analisis y consumo.
run_dbt_trusted_task = BashOperator(
    task_id = 'run_dbt_trusted',
    bash_command = 'set -x; cd /opt/airflow/dbt && dbt run --select tag:
                    trusted',
)

```

Ejecutamos modelos BDT desde Airflow utilizando comandos de terminal. Con los modelos *staging* se generan las transformaciones correspondientes a la capa Silver, mientras que los modelos etiquetados como *trusted* producen los conjuntos de datos finales que conforman la capa Gold.

6. Finalmente, dependencias:

```

## Definicion del flujo de dependencias.
# Establece el orden de ejecuci n de las tareas dentro del DAG.
[extract_raw_data_task, create_raw_schema_task] >> create_raw_table_task
create_raw_table_task >> load_raw_data_task >> run_dbt_staging_task
run_dbt_staging_task >> run_dbt_trusted_task

```

Las dependencias establecen el orden de ejecución de las tareas dentro del DAG. Gracias a esta configuración Airflow garantiza que cada etapa se ejecute **únicamente** cuando la etapa anterior haya finalizado correctamente.

Una vez tengamos todo esto, podemos ejecutar el código ya modificado para poder crear nuestra tabla y poder visualizarlo en el `localhost`. Si todo lo hicimos de la manera correcta, debemos ver todos nuestros semaforo en verde así como el *graph* bien ejecutado.

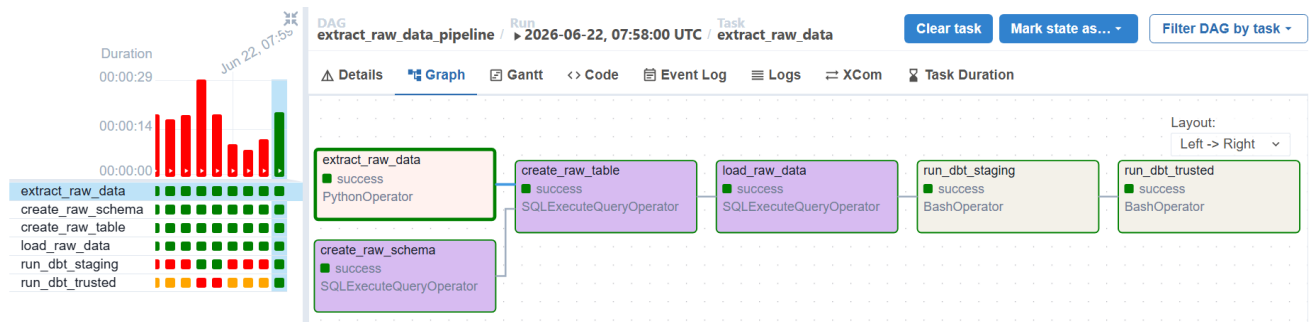


Figura 9: DAG: `extract_raw_data_pipeline`

Es completamente normal que tengamos intentos erróneos antes de ver el semáforo completamente en verde, es parte del proceso de aprendizaje; poder resolver este tipo de problemas es lo que hace que realmente te *familiarices* con el tema.

Y es importante solucionar estos problemas y obtener el semáforo verde antes de pasar a trabajar en la nube porque sino, nos terminaríamos nuestros *créditos gratis* de la misma. Es por esto que las siguientes dos sesiones se ocuparon para resolver dudas y/o problemas.

Ahora, conectemos nuestra base de datos a **pgAdmin 4**, así podemos validar los resultados en la base de datos. Para esto, creamos un nuevo server en **pgAdmin 4** el cual nosotros llamaremos **airflow** y llevará la siguiente configuración:

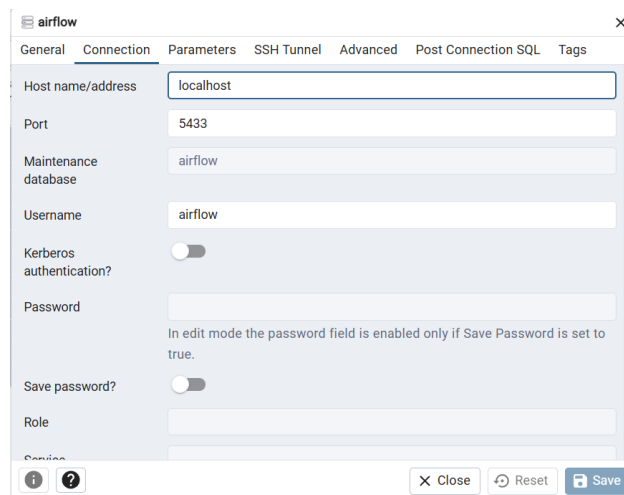


Figura 10: airflow server propiedades